

Three-Zone Compaction

A Gradient Strategy for
Long-Running AI Agent Sessions

99degree

agent-code (Avala AI) | Termux/Android

The Problem

Large language model (LLM) agents operating in extended coding sessions face a fundamental constraint: the context window. Every file read, shell command, search result, and code edit accumulates as tokens in the conversation history. When the total approaches the model's context limit, the agent must discard information to continue operating.

Naive approaches--dropping the oldest messages, truncating tool outputs, or summarizing everything--each have significant drawbacks. Dropping messages loses critical decisions. Truncation severs cause-and-effect chains. Full summarization burns expensive LLM tokens on every compaction cycle and often produces lossy summaries that omit the nuances a coding agent needs.

This essay describes a three-zone compaction strategy that balances information density, freshness, and computational cost for long-running agentic sessions.

The Three-Zone Model

When the conversation history exceeds the compaction threshold, the system divides the message history into three concentric zones of decreasing fidelity:

[===== LLM SUMMARY (50%) =====][=== MICROCOMPACT (25%) ===][=== FRESH (25%) ===]		
Oldest messages	Middle messages	Newest messages
Replaced with LLM-generated summary. Full conversation context collapsed into a dense paragraph.	Tool results cleared, conversation text preserved. message structure intact. Text-only bridge zone.	Completely untouched. All tool results, file contents, and command outputs live.

Zone 1: LLM Summary (Oldest 50%)

The oldest half of the conversation is serialized and sent to the LLM with a structured summarization prompt. The LLM produces a concise summary preserving:

- Key architectural decisions and their rationale
- Files created, modified, or deleted
- Errors encountered and how they were resolved
- The user's stated goals and constraints

This summary replaces the entire first half. The serialization includes tool call names and truncated

tool results (capped at 2,000 characters), giving the summarizer enough context to understand **what actions were taken** rather than just **what was discussed**.

The summary is inserted as a special ``is_compact_summary`` user message, preceded by a system-level boundary marker. This allows downstream code to detect compaction artifacts and handle them appropriately (e.g., preventing the summary from being double-counted in token estimates).

Zone 2: Microcompact (Middle 25%)

The middle quarter receives a lighter treatment: all tool result content is replaced with ``[Old tool result cleared]``, but the surrounding conversation text--user prompts, assistant reasoning, tool call parameters--remains intact.

This zone serves as a **textual bridge** between the dense summary and the fully-fresh recent history. The agent retains awareness of what was discussed, what files were mentioned, and what approach was being followed, even though the raw tool outputs (file contents, command stdout, search results) are gone.

Microcompaction is computationally free--no LLM call is needed. It operates by scanning for ``ToolResult`` content blocks whose corresponding ``ToolUse`` calls targeted compactable tools (``FileRead``, ``Bash``, ``Grep``, ``Glob``, ``FileEdit``, ``FileWrite``) and replacing their content with the placeholder string.

Zone 3: Fresh (Newest 25%)

The most recent quarter of the conversation is passed through completely untouched. Every tool result, file content, command output, and search result is preserved at full fidelity.

This is the zone the agent works in most actively. When it reads a file, the full contents are available for the next tool call without re-reading. When it runs a test, the full output persists for follow-up edits. The agent's working memory--its most recent concrete observations--remains intact.

Why 50/25/25?

The proportions are chosen to satisfy three competing constraints:

- 1. Token budget recovery.** Compaction exists to bring the total token count below the context window threshold. Summarizing 50% of messages replaces potentially tens of thousands of tokens

with a single ~2,000-token summary. This provides the majority of the token savings.

2. Continuity across compaction boundaries. If the summary covered more than 50%, the agent would lose too much specific context about what happened in the middle of the session. If it covered less, the token savings would be insufficient and compaction would need to fire again too soon. The 50% threshold typically frees enough tokens to sustain another full compaction cycle's worth of interaction.

3. Freshness gradient. The 25/25 split between microcompact and fresh zones means the agent always has a substantial pool of fully-fresh context (the last ~25% of messages at full fidelity), plus a text-only bridge zone that preserves conversational flow without the weight of tool outputs. This gradient--dense summary ? text bridge ? fresh data--mirrors how human memory works: we remember the gist of old events, retain the narrative of recent ones, and have perfect recall of what just happened.

Comparison with Alternative Strategies

Flat Truncation

The simplest approach: drop the oldest N messages. This is what many chat applications do. The problem for coding agents is that a file read performed 20 messages ago may still be relevant--the agent might need to cross-reference it with a later edit. Truncation provides no way to recover this information.

Full LLM Summarization

Some systems summarize the *entire* conversation history on each compaction. This is expensive (the summarization prompt grows linearly with history length) and produces a single-fidelity summary that loses the gradient between old and recent context.

pi.dev's Compaction

The pi-agent (pi.dev) system uses a similar philosophy with ``reserveTokens`` (16,384) and ``keepRecentTokens`` (20,000) thresholds, plus file-operation tracking across compactions. Their system also supports branch summarization for non-linear session trees. The three-zone model shares the insight that recent messages should be preserved at higher fidelity, but implements it as an explicit gradient rather than a binary keep/discard boundary.

Microcompact-Only

agent-code's original microcompact-only approach (clearing old tool results before each query) preserves text but never reduces the total message count. For short sessions this suffices, but over hours of agentic work the message count grows without bound until the context window overflows.

Implementation Details

Pre-Query vs. Full Compaction

The three-zone strategy is applied during **LLM-triggered compaction**--the full compaction that fires when estimated tokens approach the context window limit. Before this fires, a lighter **microcompact** step runs on every query, clearing old tool results while keeping the 5 most recent tool outputs intact. This pre-query microcompact acts as a first line of defense, freeing tokens without the cost of an LLM call.

The flow is:

```
User sends message
  ? Pre-query microcompact (keep_recent=5) -- free tokens cheaply
    ? Still over threshold?
      ? 3-zone LLM compaction (50/25/25)
        ? Still over threshold?
          ? Context collapse (snip middle messages)
            ? Still over threshold?
              ? Error surfaced to user
```

Cache Awareness

Modern LLM APIs (Anthropic, OpenAI) use prompt caching: if the beginning of a prompt matches a previously-seen prefix, the Key-Value representations are reused from cache, saving both cost and latency. Microcompact modifies message content in place, which invalidates the KV cache for everything after the modification point. This is an accepted tradeoff--the token savings from microcompact are necessary to stay within the context window--but it means that compaction cycles incur a full prefill cost on the next API call.

Serialization for the Summarizer

The `build_compact_summary_prompt` function converts the conversation into a flat text format optimized for the summarizer LLM:

- User messages: User: `<text>`
- Assistant messages: Assistant: `<text>`
- Tool calls: `ToolCall(<id>): [<name>] <truncated input>`
- Tool results: `ToolResult(<id>): [<name>] <first 2000 chars>`

All content is run through a secret masker before serialization, ensuring that API keys or credentials that appeared in tool output never leak into the summary.

Cut Point Snapping

The split point between Zone 1 (summarized) and Zone 2 (microcompacted) is snapped to the nearest user message boundary. This ensures the summary prompt begins with a user turn, which produces more coherent summaries from the LLM.

Behavioral Impact

For a typical coding session with a 200K-token context window:

- **Before compaction:** ~190K tokens (approaching threshold)
- **After one compaction cycle:**
 - Zone 1 (50%): ~95K tokens ? ~2K token summary
 - Zone 2 (25%): ~47K tokens ? ~5K tokens (tool results cleared)
 - Zone 3 (25%): ~47K tokens ? ~47K tokens (untouched)
- **Total:** ~54K tokens (72% reduction)

This leaves ~146K tokens of headroom before the next compaction, typically sustaining 30-60 minutes of active agentic work before compaction is needed again.

Conclusion

Long-running AI coding agents need compaction strategies that respect the gradient of information relevance. The three-zone model--summarize the old, thin the middle, preserve the new--provides a stable, predictable mechanism for managing context across compaction cycles. It is computationally efficient (one LLM call per cycle for the summary, plus free microcompaction), information-preserving

(the agent retains text-level awareness of the full session), and fresh (the most recent quarter is always at full fidelity).

This approach treats context window management not as an emergency measure but as a first-class architectural concern--the way a well-designed operating system manages memory through tiers of cache, RAM, and disk.